
String Data Structure in Java

- 01. Introduction to strings*
- 02. Mutability and Immutability objects*
- 03. Heap and SCP memory areas*
- 04. java.lang.String constructors and methods*
- 05. java.lang.StringBuffer constructors and methods*
- 06. java.lang.StringBuilder constructors and methods*
- 07. StringBuffer vs StringBuilder*
- 08. java.util.StringTokenizer constructors and methods*
- 09. Regular Expression and Applications*
- 10. Programs on String*
- 11. Programs on StringBuffer and StringBuilder*
- 12. Programs on StringTokenizer*

01. Introduction to strings

like other programming languages, collection or group or sequence of characters is called as string. strings are enclosed within double quotes "like this". To perform basic string level operations java has provide predefined class in lang and util packages. java strings are divided into the following four types

- 1. java.lang.String*
- 2. java.lang.StringBuffer*
- 3. java.lang.StringBuilder*
- 4. java.util.StringTokenizer*

02. Mutability and Immutability objects

String class objects are immutable objects, if we are trying to perform any modifications on the string object, with those modifications a new string object will be created but old content can't be modified. i.e. modifications are not allowed.

Ex: String

if we are trying to perform any modifications on the existing object, if those modifications will be performed on the same object, then such type of objects are called as mutable objects. i.e. modifications are allowed.

Ex: StringBuffer and StringBuiler

Ex:

class Test

```
{  
  
    public static void main(String[] args)  
  
    {  
  
        String s = new String("abc");  
  
        s.concat("def");  
  
        System.out.println(s);//abc  
  
    }  
  
}
```

Ex:

```
class Test  
  
{  
  
    public static void main(String[] args)  
  
    {  
  
        StringBuffer sb = new StringBuffer("wx");  
  
        sb.append("yz");  
  
        System.out.println(sb);//wxyz  
  
    }  
  
}
```

== is meant for content comparing for primitives

== is meant for reference or address comparing for objects

.equals() is meant for reference or address comparing for objects (java.lang.Object)

Note1:

== is meant for address of reference comparision, .equals() method is overriden in java.lang.String class for content compaision.

Ex:

class Test

```
{  
  
    public static void main(String[] args)  
    {  
  
        String s1 = new String("abc");  
  
        String s2 = new String("abc");  
  
        System.out.println(s1==s2);//false  
  
        System.out.println(s1.equals(s2));//true  
  
    }  
}
```

Note1:

== is meant for address of reference comparision, .equals() method is not overriden in java.lang.StringBuffer class for content compaision, hence it is also meant for address comparision

Ex:

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        StringBuffer sb1 = new StringBuffer("wxyz");
```

```
        StringBuffer sb2 = new StringBuffer("wxyz");
```

```
        System.out.println(sb1==sb2);//false
```

```
        System.out.println(sb1.equals(sb2));//false
```

```
    }
```

```
}
```

03. Heap and SCP memory areas

Ex1:

```
String s = new String("prakash");
```

heap -----> 1

scp -----> 1

total ----> 2

In this case two objects are created one is in heap and other one is in scp (string constant pool) and "s" is always pointing to heap object.

Ex2:

String s = "prakash";

heap -----> 0

scp -----> 1

total -----> 1

In this case only one object will be created in scp area.

Ex3:

String s1 = new String("abc");

String s2 = new String("abc");

String s3 = "abc";

String s4 = "abc";

heap -----> 2

scp -----> 1

total -----> 3

=> Object creation in SCP is always optional, first JVM will check whether the content is already existed in SCP are not, if it is existed same reference will be reused.

Ex4:

```
String s = new String("abc");
```

```
s.concat("def");
```

```
s=s.concat("wxyz");
```

heap ----> 3

scp -----> 3

total ---> 6

Ex5:

```
String s1 = new String("spring");
```

```
s1.concat("fall");
```

```
String s2 = s1.concat("winter");
```

```
s2.concat("summer");
```

heap ----> 4

scp -----> 4

total ---> 8

Ex:

class Test

```
{  
  
    public static void main(String[] args)  
  
    {  
  
        String s1 = new String("spring");  
  
        s1.concat("fall");  
  
        String s2 = s1.concat("winter");  
  
        s2.concat("summer");  
  
        System.out.println(s1);//spring  
  
        System.out.println(s2);//springwinter  
  
    }  
}
```

Ex6:

```
String s1 = new String("i love my java");  
String s2 = new String("i love my java");  
String s3 = "i love my java";  
String s4 = "i love my java";  
String s5 = "i love "+"my java";  
String s6 = "i love ";  
String s7 = s6 + "my java";  
final String s8 = "i love ";  
String s9 = s8 + "my java";
```


heap ----> 3

scp -----> 3

total ---> 6

Ex:

class Test

{

 public static void main(String[] args)

 {

 String s1 = new String("i love my java");

 String s2 = new String("i love my java");

 System.out.println(s1==s2);//false

 String s3 = "i love my java";

 String s4 = "i love my java";

 System.out.println(s1==s3);//false

 System.out.println(s2==s3);//false

 System.out.println(s3==s4);//true

 String s5 = "i love "+"my java";

 System.out.println(s3==s5);//true

 System.out.println(s4==s5);//true

```
String s6 = "i love ";  
String s7 = s6 + "my java";  
System.out.println(s4==s7);//false
```

```
final String s8 = "i love ";  
System.out.println(s6==s8);//true
```

```
String s9 = s8 + "my java";  
System.out.println(s4==s9);//true
```

```
}
```

```
}
```

Ex7:

```
String s1 = "abc";  
String s2 = s1.toUpperCase();  
String s3 = s1.toLowerCase();
```

heap ----> 1

scp -----> 1

total ---> 2

Ex:

class Test

{

public static void main(String[] args)

{

String s1 = "abc";

String s2 = s1.toUpperCase();

String s3 = s1.toLowerCase();

System.out.println(s1==s2);//false

System.out.println(s1==s3);//true

}

}

Ex8:

String s1 = "abc";

String s2 = s1.toString();

heap ----> 0

scp -----> 1

total ---> 1

Ex:

```
class Test

{

    public static void main(String[] args)

    {

        String s1 = "abc";

        String s2 = s1.toString();


        System.out.println(s1==s2);//true

    }

}
```

Ex9:

```
String s1 = new String("abc");

String s2 = s1.toString();
```

heap ---> 1

scp ----> 1

total --> 2

class Test

```
{

    public static void main(String[] args)
```

```
{  
  
    String s1 = new String("abc");  
  
    String s2 = s1.toString();  
  
    System.out.println(s1==s2);//true  
  
}  
}
```

Ex10:

```
String s1 = new String("abc");  
String s2 = s1.toString();  
String s3 = s1.toUpperCase();  
String s4 = s1.toLowerCase();  
String s5 = s1.toUpperCase();  
String s6 = s3.toLowerCase();
```

heap ----> 4

scp -----> 1

total ---> 5

class Test

```
{
```

```
public static void main(String[] args)
{
    String s1 = new String("abc");
    String s2 = s1.toString();
    System.out.println(s1==s2);//true

    String s3 = s1.toUpperCase();
    String s4 = s1.toLowerCase();
    System.out.println(s1==s4);//true

    String s5 = s1.toUpperCase();
    System.out.println(s3==s5);//false

    String s6 = s3.toLowerCase();
    System.out.println(s1==s6);//false
}
}
```

java.lang.String constructors and methods

String()

it is used to create an empty string object.

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        String s = new String();

        System.out.println(s);

        System.out.println(s.length());

        System.out.println(s.isEmpty());

    }

}
```

String(string)

it is used to create a string object with given string literal value.

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        String s = new String("vishal");

    }

}
```

```
System.out.println(s);//vishal
```

```
System.out.println(s.length());//6
```

```
System.out.println(s.isEmpty());//false
```

```
}
```

```
}
```

String(char[])

it creates a new string object with given characters.

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        char[] ch = {'w','e','l','c','o','m','e'};
```

```
        String s = new String(ch);
```

```
        System.out.println(s);//welcome
```

```
    }
```

```
}
```

String(char[],int start_index,int num_of_chars)

it extract the characters from start_index to number of characters

Ex:

```
import java.util.*;

class Test
{
    public static void main(String[] args)
    {
        char[] ch = {'w','e','l','c','o','m','e'};
        //      0  1  2  3  4  5  6

        String s1 = new String(ch);

        System.out.println(s1);//welcome

        String s2 = new String(ch,3,4);

        System.out.println(s2);//come

        String s3 = new String(ch,3,2);

        System.out.println(s3);//co

    }
}
```

String(byte[])

it creates a new string object with given byte array [ascii values].

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        byte[] b = {97, 98, 99, 65, 66, 67, 68};

        //      0  1  2  3  4  5  6

        String s = new String(b);

        System.out.println(s);//abcABCD

    }

}
```

String(StringBuffer)

it is used to convert our StringBuffer object into String object.

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {
```

```
StringBuffer sb = new StringBuffer("abc");

String s = new String(sb);

System.out.println(s);//abc

System.out.println(sb);//abc

    }

}
```

String(StringBuilder)

it is used to convert our StringBuilder object into String object.

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        StringBuilder sb = new StringBuilder("abcd");

        String s = new String(sb);

        System.out.println(s);//abcd

        System.out.println(sb);//abcd

    }

}
```

int length()

it returns number of characters present in the given string object.

Ex:

```
import java.util.*;

class Test
{
    public static void main(String[] args)
    {
        String s = new String("prakash babu");
        System.out.println(s);//prakash babu
        System.out.println(s.length());//12
    }
}
```

boolean isEmpty()

it returns true if the given string is empty else false.

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        System.out.println("abc".isEmpty()); //false

        System.out.println(" ".isEmpty()); //false

        System.out.println("").isEmpty()); //true

    }

}
```

boolean startsWith(string)

it returns true if the given string starts with another string else false

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        System.out.println("java is very easy".startsWith("python")); //false

        System.out.println("java is very easy".startsWith("java")); //true

    }

}
```

```
}
```

```
}
```

boolean endsWith(String)

it returns true if the given string ends with another string else false

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        System.out.println("java is very easy".endsWith("easy")); //true
```

```
        System.out.println("java is very easy".endsWith("difficult")); //false
```

```
    }
```

```
}
```

boolean equals(string)

it returns true if the given string is equal with another string, else false. consider the case while comparing.

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        System.out.println("java".equals("python")); //false
```

```
        System.out.println("java".equals("abcd")); //false
```

```
        System.out.println("java".equals("JAVA")); //false
```

```
        System.out.println("java".equals("java")); //true
```

```
    }
```

```
}
```

```
boolean equalsIgnoreCase(string)
```

it returns true if the given string is equal with another string, else false. it ignores case while comparing.

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
System.out.println("java".equals("python"));//false

System.out.println("java".equals("abcd"));//false

System.out.println("java".equalsIgnoreCase("JAVA"));//true

System.out.println("java".equals("java"));//true

    }

}
```

char[] toCharArray():

it returns char array on the given string object

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String s = "abcd";
```

```
        char[] ch = s.toCharArray();
```

```
        System.out.println(s);//abcd
```

```
        System.out.println(Arrays.toString(ch));//['a','b','c','d']
```

```
    }
```

```
}
```


byte[] getBytes()

it returns byte array on the given string object

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String s = "abcd";
```

```
        byte[] b = s.getBytes();
```

```
        System.out.println(s);//abcd
```

```
        System.out.println(Arrays.toString(b));//[97,98,99,100]
```

```
    }
```

```
}
```

char charAt(int index)

it returns char located at the given index, else AIOBE

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String s = "abcd";
```

```
        System.out.println(s);//abcd
```

```
        System.out.println(s.charAt(0));//a
```

```
        System.out.println(s.charAt(1));//b
```

```
        System.out.println(s.charAt(2));//c
```

```
        System.out.println(s.charAt(3));//d
```

```
    }
```

```
}
```

```
int indexOf(char)
```

it returns index value of the given character

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
public static void main(String[] args)
{
    String s = "abcaba";
    //    012345
    System.out.println(s);//abcaba
    System.out.println(s.indexOf('a'));//0
    System.out.println(s.indexOf('b'));//1
    System.out.println(s.indexOf('d'));//-1
}
}
```

int lastIndexOf(char)

it returns last index value of the given character

Ex:

```
import java.util.*;
class Test
{
    public static void main(String[] args)
    {
        String s = "abcaba";
        //    012345
```

```
        System.out.println(s);//abcaba

        System.out.println(s.lastIndexOf('a'));//5

        System.out.println(s.lastIndexOf('b'));//4

        System.out.println(s.lastIndexOf('d'));//-1

    }

}
```

String substring(int index)

it returns complete string starting from the given index to till end

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        String s = "java is very very easy programming";

        //      0  5 8  13 18 23

        System.out.println(s);//java is very very easy programming

        System.out.println(s.substring(5));//is very very easy programming

        System.out.println(s.substring(13));//very easy programming

        System.out.println(s.substring(23));//programming

    }

}
```

```
}
```

```
}
```

String substring(int sindex,int eindex)

it returns complete string starting from the given index to ending index-1

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String s = "java is very very easy programming";
```

```
        //      0  5 8  13 18 23
```

```
        System.out.println(s);//java is very very easy programming
```

```
        System.out.println(s.substring(5,7));//is
```

```
        System.out.println(s.substring(8,17));//very very
```

```
        System.out.println(s.substring(18,34));//easy programming
```

```
    }
```

```
}
```

String concat(string)

it is used to concatenate with another string. we can also use + operator for this purpose.

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        String s1 = "wel";

        String s2 = "come";

        System.out.println(s1);//wel

        System.out.println(s2);//come

        System.out.println(s1+s2);//welcome

        System.out.println(s1.concat(s2));//welcome

        System.out.println(s1);//wel

        System.out.println(s2);//come

    }

}
```

String replace(old_char, new_char)

it replaces old char with given new char

Ex:

```
import java.util.*;

class Test
{
    public static void main(String[] args)
    {
        String s = new String("abcaba");
        System.out.println(s);//abcaba
        System.out.println(s.replace('a','x'));//xbcxbx
    }
}
```

String toUpperCase()

it converts the given string into upper case

Ex:

```
import java.util.*;

class Test
{
    public static void main(String[] args)
```

```
{
    System.out.println("java".toUpperCase()); //JAVA
    System.out.println("JAVA".toUpperCase()); //JAVA
    System.out.println("JaVa".toUpperCase()); //JAVA
    System.out.println("JavA".toUpperCase()); //JAVA
}
```

String toLowerCase()

it converts the given string into lower case

Ex:

```
import java.util.*;

class Test
{
    public static void main(String[] args)
    {
        System.out.println("java".toLowerCase()); //java
        System.out.println("JAVA".toLowerCase()); //java
        System.out.println("JaVa".toLowerCase()); //java
        System.out.println("JavA".toLowerCase()); //java
    }
}
```


}

String[] split(delimtier)

it divides the given string into string[] based on given delimiter

Ex1:

import java.util.;*

class Test

{

public static void main(String[] args)

{

String s = "java is very easy";

String[] ss = s.split(" ");

System.out.println(s);//java is very easy

System.out.println(Arrays.toString(ss));//[java, is, very, easy]

}

}

Ex2:

import java.util.;*

class Test

```
{  
  
    public static void main(String[] args)  
  
    {  
  
        String s = "13-1-2023";  
  
        String[] ss = s.split("-");  
  
        System.out.println(s);//13-1-2023  
  
        System.out.println(Arrays.toString(ss));//[13,1,2023]  
  
    }  
  
}
```

java.lang.StringBuffer constructors and methods

if the content is fixed, not changing frequently then we should go for String.

if the content is not fixed and keep on changing then we should go for StringBuffer

String ---> immutable

StringBuffer ---> mutable

StringBuffer()

it creates a new string buffer object with default capacity as 16.

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        StringBuffer sb = new StringBuffer();

        System.out.println(sb);//

        System.out.println(sb.length());//0

        System.out.println(sb.capacity());//16

    }

}
```

StringBuffer(String)

it creates a new string buffer object with given string and capacity = len(s)+16

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {
```

```
StringBuffer sb = new StringBuffer("abc");

System.out.println(sb);//abc

System.out.println(sb.length());//3

System.out.println(sb.capacity());//3+16=19

    }

}
```

StringBuffer(int capacity)

it creates a new string buffer object with given capacity

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        StringBuffer sb = new StringBuffer(50);

        System.out.println(sb);//

        System.out.println(sb.length());//0

        System.out.println(sb.capacity());//50

    }

}
```

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        StringBuffer sb = new StringBuffer();
```

```
        System.out.println(sb);//
```

```
        System.out.println(sb.length());//0
```

```
        System.out.println(sb.capacity());//16
```

```
        sb.append("abcdefghijklmnop");
```

```
        System.out.println(sb);//abcdefghijklmnop
```

```
        System.out.println(sb.length());//16
```

```
        System.out.println(sb.capacity());//16
```

```
        sb.append("q");
```

```
        System.out.println(sb);//abcdefghijklmnopq
```

```
        System.out.println(sb.length());//17
```

```
        System.out.println(sb.capacity());//(16+1)*2=17*2=34
```

```
        //newcapacity = (oldcapacity+1)*2
```

```
        sb.append("abcdefghijklmnopq");
```

```
        System.out.println(sb);//abcdefghijklmnopqabcdefghijklmnopq
```

```
        System.out.println(sb.length());//34
```

```
System.out.println(sb.capacity()); //34

sb.append("wxyz");

System.out.println(sb); //abcdefghijklmnpqabcdefghijklmnpqwxyz

System.out.println(sb.length()); //38

System.out.println(sb.capacity()); // (34+1)*2=35*2=70

    }

}
```

int length()

it returns number of characters present in StringBuffer.

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        StringBuffer sb = new StringBuffer("abcd");

        System.out.println(sb); //abcd

        System.out.println(sb.length()); //4

    }

}
```

}

int capacity()

it returns max number of characters it can hold.

Ex:

import java.util.;*

class Test

{

public static void main(String[] args)

{

StringBuffer sb = new StringBuffer("abcd");

System.out.println(sb);//abcd

System.out.println(sb.length());//4

System.out.println(sb.capacity());//4+16=20

}

}

void ensureCapacity(int capacity)

it is used to increase the capacity of string buffer object

Ex:

```
import java.util.*;

class Test
{
    public static void main(String[] args)
    {
        StringBuffer sb = new StringBuffer("abcd");

        System.out.println(sb);//abcd

        System.out.println(sb.length());//4

        System.out.println(sb.capacity());//4+16=20

        sb.ensureCapacity(50);

        System.out.println(sb.capacity());//50
    }
}
```

`void setLength(int length)`

it sets the length of string buffer object and removes extra content.

Ex:

```
import java.util.*;

class Test
```



```
{  
  
    public static void main(String[] args)  
  
    {  
  
        StringBuffer sb = new StringBuffer("abcd");  
  
        System.out.println(sb);//abcd  
  
        System.out.println(sb.length());//4  
  
        System.out.println(sb.capacity());//4+16=20  
  
        sb.setLength(2);  
  
        System.out.println(sb);//ab  
  
        System.out.println(sb.length());//2  
  
        System.out.println(sb.capacity());//20  
  
    }  
}
```

void trimToSize()

it finalize the given string with number of characters existed

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
{  
  
    StringBuffer sb = new StringBuffer("abcd");  
  
    System.out.println(sb);//abcd  
  
    System.out.println(sb.length());//4  
  
    System.out.println(sb.capacity());//4+16=20  
  
    sb.setLength(2);  
  
    System.out.println(sb);//ab  
  
    System.out.println(sb.length());//2  
  
    System.out.println(sb.capacity());//20  
  
    sb.trimToSize();  
  
    System.out.println(sb);//ab  
  
    System.out.println(sb.length());//2  
  
    System.out.println(sb.capacity());//2  
  
}  
  
}
```

char charAt(int index)

it returns character located at the given index value.

Ex:

import java.util.;*

class Test

```
{  
  
    public static void main(String[] args)  
  
    {  
  
        StringBuffer sb = new StringBuffer("abcd");  
  
        System.out.println(sb);//abcd  
  
        System.out.println(sb.charAt(0));//a  
  
        System.out.println(sb.charAt(1));//b  
  
        System.out.println(sb.charAt(2));//c  
  
        System.out.println(sb.charAt(3));//d  
  
    }  
}
```

void setCharAt(int index,char ch)

it perform replacement operation on the given string buffer

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
        StringBuffer sb = new StringBuffer("welkome");
```

```
        System.out.println(sb);//welkome

        sb.setCharAt(3,'c');

        System.out.println(sb);//welcome

    }

}
```

void deleteCharAt(int index)

it removes the character located at the given index

Ex:

```
import java.util.*;

class Test

{

    public static void main(String[] args)

    {

        StringBuffer sb = new StringBuffer("welkome");

        System.out.println(sb);//welkome

        sb.deleteCharAt(3);

        System.out.println(sb);//welome

    }

}
```

StringBuffer append(object)

it appends i.e. adds the given object at the end of string buffer

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        StringBuffer sb = new StringBuffer();
```

```
        System.out.println(sb);//
```

```
        sb.append("welcome ");
```

```
        System.out.println(sb);//welome
```

```
        sb.append("python ");
```

```
        System.out.println(sb);//welome python
```

```
        sb.append("programming");
```

```
        System.out.println(sb);//welome python programming
```

```
    }
```

```
}
```

StringBuffer insert(int index,object)

it inserts the given object at the specified location.

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        StringBuffer sb = new StringBuffer();
```

```
        System.out.println(sb);//
```

```
        sb.append("welcome ");
```

```
        System.out.println(sb);//welome
```

```
        sb.append("python ");
```

```
        System.out.println(sb);//welome python
```

```
        sb.append("programming");
```

```
        sb.insert(7," to");
```

```
        System.out.println(sb);//welome to python programming
```

```
        sb.insert(10," java and");
```

```
        System.out.println(sb);//welome to java and python programming
```

```
    }
```

```
}
```

StringBuffer delete(int sindex,int eindex)

it removes the characters from s index to ending index

Ex:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        StringBuffer sb = new StringBuffer();
```

```
        System.out.println(sb);//
```

```
        sb.append("welcome ");
```

```
        System.out.println(sb);//welome
```

```
        sb.append("python ");
```

```
        System.out.println(sb);//welome python
```

```
        sb.append("programming");
```

```
        sb.insert(7," to");
```

```
        System.out.println(sb);//welome to python programming
```

```
        sb.insert(10," java and");
```

```
        System.out.println(sb);//welome to java and python programming
```

```
        sb.delete(15,26);
```

```
        System.out.println(sb);//welome to java programming
```

```
    }
```

}

StringBuffer reverse()

it reverse the given string buffer's content

Ex:

import java.util.;*

class Test

{

public static void main(String[] args)

{

StringBuffer sb = new StringBuffer("abcdefgh");

System.out.println(sb);//abcdefgh

sb.reverse();

System.out.println(sb);//hgfedcba

}

}

java.lang.StringBuilder constructors and methods

it is exactly same as java.lang.StringBuffer, except the following differences

StringBuffer vs StringBuilder

StringBuffer

- => 1.0 version*
- => synchronized*
- => only one thread is allowed*
- => sequential execution*
- => increase application time*
- => performance improved*
- => thread safe*
- => deprecated/outdated*

StringBuilder

- => 1.5 version*
- => non-synchronized*
- => multiple threads are allowed*
- => parallel execution*
- => decreases application time*
- => performance not that good*
- => not thread safe*
- => not deprecated/outdated*

java.util.StringTokenizer constructors and methods

it is an utility class existed in java.util package and StringTokenizer is used to divide the given string into tokens.

"java is very easy" -----> java, is, very, easy

"19-12-2002" -----> 19, 12, 2002

"09:30:45" -----> 09, 30, 45

StringTokenizer st = new StringTokenizer(String)

StringTokenizer st = new StringTokenizer(String,delimiter)

int countTokens() -----> returns number of tokens

boolean hasMoreTokens() -> returns true if there is a token

String nextToken() -----> returns current token and transfer the control to next

Ex:

import java.util.;*

class Test

{

public static void main(String[] args)

{

String s = "java is very easy";

StringTokenizer st = new StringTokenizer(s);

```
System.out.println(st.countTokens()); //4

while(st.hasMoreTokens()){

    System.out.println(st.nextToken());

}
```

```
}
```

output:

java

is

very

easy

Ex2:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String s = "13-1-2023";
```

```
        StringTokenizer st = new StringTokenizer(s, "-");
```

```
        System.out.println(st.countTokens()); //3
```

```
        while(st.hasMoreTokens()){
```

```
System.out.println(st.nextToken());
```

```
}
```

```
}
```

```
}
```

output:

3

13

1

2023

Ex3:

```
import java.util.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String s = "09:30:34";
```

```
        StringTokenizer st = new StringTokenizer(s, ":");
```

```
        System.out.println(st.countTokens()); //3
```

```
        while(st.hasMoreTokens()){
```

```
            System.out.println(st.nextToken());
```

```
}
```

```
}
```

```
}
```

output:

3

09

30

34

Regular Expression and Applications

a group of strings according to a particular format or pattern or template is called as regular expression.

steps to prepare re object

1) import java.util.regex.;*

java.util -> it imports all the classes and interfaces from util pkg not sub-pkgs

java.util.regex --> it imports all the classes and interfaces from regex sub-pkg

2) pattern object (format)

3) matcher object (target)

mobile number validation:

123 *invalid*

1234567890 *invalid*

7386237319 *valid*

1386237319 *invalid*

format or re: [6-9][0-9]{9}

Ex:

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Pattern p = Pattern.compile("[0-9]");//re
```

```
        Matcher m = p.matcher("a1b2c9d57u");//target string
```

```
        int c=0;
```

```
        while(m.find()){
```

```
            System.out.println(m.start()+"====> "+m.end()+"====>
```

```
            "+m.group());
```

```
        }
    }
}
```

```
}  
  
    System.out.println("count="+c);  
  
}  
  
}
```

1 =====> 2 =====> 1

3 =====> 4 =====> 2

5 =====> 6 =====> 9

7 =====> 8 =====> 5

8 =====> 9 =====> 7

count=5

Ex:

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Pattern p = Pattern.compile("[a-z]");//re
```

```
        Matcher m = p.matcher("a1b2c9d57u");//target string
```

```
        int c=0;
```

```
while(m.find()){

    System.out.println(m.start()+"====> "+m.end()+"====>
"+m.group());

    c++;

}

System.out.println("count="+c);

}

}
```

output:

0====> 1====> a

2====> 3====> b

4====> 5====> c

6====> 7====> d

9====> 10====> u

count=5

predefined character classes:

~~~~~

\\s    *space characters*

\\S    *except space character*

\\d    *digits*

\\D    *except digits*

\\w    *word character i.e. a-z or A-Z or 0-9*



`\\W` except word character (special characters)

`.` all characters

Ex:

----

```
Pattern p = Pattern.compile("\\s");
```

```
Pattern p = Pattern.compile("\\S");
```

```
Pattern p = Pattern.compile("\\d");
```

```
Pattern p = Pattern.compile("\\D");
```

```
Pattern p = Pattern.compile("\\w");
```

```
Pattern p = Pattern.compile("\\W");
```

```
Pattern p = Pattern.compile(".");
```

Ex:

---

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Pattern p = Pattern.compile("\\W");//re
```

```
        Matcher m = p.matcher("ab c$123#iJk^45 6*pQr @ wXYZ");
```

```
        int c=0;
```

```

while(m.find()){

    System.out.println(m.start()+"====> "+m.end()+"====> "+m.group());

    c++;

}

System.out.println("count="+c);

}

}

```

## *userdefined character classes*

~~~~~

<code>[abc]</code>	<i>either 'a' or 'b' or 'c'</i>
<code>[^abc]</code>	<i>except either 'a' or 'b' or 'c'</i>
<code>[0-9]</code>	<i>all digits from 0 to 9</i>
<code>[^0-9]</code>	<i>except digits from 0 to 9</i>
<code>[a-z]</code>	<i>lower case characters</i>
<code>[^a-z]</code>	<i>except lower case characters</i>
<code>[A-Z]</code>	<i>upper case characters</i>
<code>[^A-Z]</code>	<i>except upper case characters</i>
<code>[a-zA-Z]</code>	<i>all lower case and upper case characters</i>
<code>[^a-zA-Z]</code>	<i>except lower case and upper case characters</i>
<code>[a-zA-Z0-9]</code>	<i>all lower case, upper case and digits</i>
<code>[^a-zA-Z0-9]</code>	<i>except lower case, upper case and digits</i>

```
import java.util.*;

import java.util.regex.*;

class Test

{

    public static void main(String[] args)

    {

        Pattern p = Pattern.compile("[^a-zA-Z0-9 ]");//re

        Matcher m = p.matcher("ab c$123#iJk^45 6*pQr @ wXYZ");

        int c=0;

        while(m.find()){

            System.out.println(m.start()+"====> "+m.end()+"====> "+m.group());

            c++;

        }

        System.out.println("count="+c);

    }

}
```

quantifiers:

~~~~~

*a* exactly one *a*

*a\** zero or more number of *a*'s

*a+* one or more number of *a*'s

*a?* zero or one *a*

*a{m}* exactly '*m*' number of *a*'s

*a{m,n} min 'm' number of a's and max 'n' number of a's*

Ex:

---

```
import java.util.*;
import java.util.regex.*;

class Test
{
    public static void main(String[] args)
    {
        Pattern p = Pattern.compile("a{3,5}");//re
        Matcher m = p.matcher("abaabaaabaaaabaaaaabaaaaaab");
        int c=0;
        while(m.find()){
            System.out.println(m.start()+"====> "+m.end()+"====> "+m.group());
            c++;
        }
        System.out.println("count="+c);
    }
}
```

25. Impl prg to validate ATM pin number.

-----

```
import java.util.*;
```

```
import java.util.regex.*;

class Test

{

    public static void main(String[] args)

    {

        Scanner obj = new Scanner(System.in);

        String s = obj.nextLine();

        System.out.println(s.matches("[0-9]{4}"));

    }

}
```

C:\3pm>javac Test.java

C:\3pm>java Test

1234

true

C:\3pm>java Test

123

false

C:\3pm>java Test

12345

false

*C:\3pm>java Test*

*a1b2*

*false*

*26. Impl prg to validate mobile number*

```
-----  
  
import java.util.*;  
  
import java.util.regex.*;  
  
class Test  
{  
  
    public static void main(String[] args)  
    {  
  
        Scanner obj = new Scanner(System.in);  
  
        String s = obj.nextLine();  
  
        System.out.println(s.matches("[6-9][0-9]{9}"));  
  
    }  
}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*7386237319*

*true*

```
C:\3pm>java Test
```

```
1386237319
```

```
false
```

```
C:\3pm>java Test
```

```
73862373190
```

```
false
```

27. Impl prg to validate gmail id

-----

```
[a-z][a-z0-9_][a-z0-9_]+@gmail[.]com
```

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner obj = new Scanner(System.in);
```

```
        String s = obj.nextLine();
```

```
        System.out.println(s.matches("[a-z][a-z0-9_][a-z0-9_]+@gmail[.]com"));
```

```
    }
```

```
}
```

```
C:\3pm>javac Test.java
```

```
C:\3pm>java Test
```

```
a@gmail.com
```

```
false
```

```
C:\3pm>java Test
```

```
ab@gmail.com
```

```
false
```

```
C:\3pm>java Test
```

```
abc@gmail.com
```

```
true
```

```
C:\3pm>java Test
```

```
abc123@gmail.com
```

```
true
```

```
C:\3pm>java Test
```

```
123abc@gmail.com
```

```
false
```

```
C:\3pm>java Test
```



*Abc@gmail.com*

*false*

*C:\3pm>java Test*

*abcd123@yahoo.com*

*false*

*28. Impl prg to validate student university hall ticket number*

-----

*DSxxxx ==> DS[0-9]{4}*

*import java.util.\*;*

*import java.util.regex.\*;*

*class Test*

*{*

*public static void main(String[] args)*

*{*

*Scanner obj = new Scanner(System.in);*

*String s = obj.nextLine();*

*System.out.println(s.matches("DS\\d{4}"));*

*}*

*}*

*C:\3pm>javac Test.java*

```
C:\3pm>java Test
```

```
DS1234
```

```
true
```

```
C:\3pm>java Test
```

```
DS12345
```

```
false
```

```
C:\3pm>java Test
```

```
TS1234
```

```
false
```

29. Impl prg to validate bike registration number

-----

```
TS 21 AB 1234
```

```
TS[12][0-9][A-Z]{2}[0-9]{4}
```

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
{  
  
    Scanner obj = new Scanner(System.in);  
  
    String s = obj.nextLine();  
  
    System.out.println(s.matches("TS[12][0-9][A-Z]{2}[0-9]{4}"));  
  
}  
  
}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*TS22AB1234*

*true*

*C:\3pm>java Test*

*TS90AB1234*

*false*

*30. Impl prg to validate given date.*

-----

*20-01-2023*

*[0123][0-9]-[01][0-9]-202[3-9]*

*import java.util.\*;*

*import java.util.regex.\*;*

*class Test*

```
{  
  
    public static void main(String[] args)  
  
    {  
  
        Scanner obj = new Scanner(System.in);  
  
        String s = obj.nextLine();  
  
        System.out.println(s.matches("[0123][0-9]-[01][0-9]-202[3-9]"));  
  
    }  
  
}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*20-01-2023*

*true*

*C:\3pm>java Test*

*45-01-2023*

*false*

*C:\3pm>java Test*

*39-01-2023*

*true*

*C:\3pm>java Test*

*20-90-2023*

*false*

*C:\3pm>java Test*

*20-01-2022*

*false*

*01. Impl prg to read str and print char and corresponding index value.*

```
-----  
  
import java.util.*;  
  
import java.util.regex.*;  
  
class Test  
{  
  
    public static void main(String[] args)  
    {  
  
        Scanner obj = new Scanner(System.in);  
  
        String s = obj.nextLine();  
  
        System.out.println(s);  
  
        for(int i=0;i<s.length();i++){  
  
            System.out.println("index= "+i+" and char= "+s.charAt(i));  
  
        }  
  
    }  
  
}
```

```
C:\3pm>javac Test.java
```

```
C:\3pm>java Test
```

```
programming
```

```
programming
```

```
index= 0 and char= p
```

```
index= 1 and char= r
```

```
index= 2 and char= o
```

```
index= 3 and char= g
```

```
index= 4 and char= r
```

```
index= 5 and char= a
```

```
index= 6 and char= m
```

```
index= 7 and char= m
```

```
index= 8 and char= i
```

```
index= 9 and char= n
```

```
index= 10 and char= g
```

02. Impl prg to read str and print chars present at even/odd index values.

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
public static void main(String[] args)
{
    Scanner obj = new Scanner(System.in);
    String s = obj.nextLine();
    System.out.println(s);
    for(int i=0;i<s.length();i++){
        if(i%2==0)//i%2!=0
            System.out.println("index= "+i+" and char= "+s.charAt(i));
    }
}
```

C:\3pm>javac Test.java

C:\3pm>java Test

python

python

index= 0 and char= p

index= 2 and char= t

index= 4 and char= o

03. Impl prg to print vowels/consonants present in the given str.

-----  
import java.util.\*;

```
import java.util.regex.*;

class Test
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        String s = obj.nextLine();
        System.out.println(s);
        for(int i=0;i<s.length();i++){
            char ch = s.charAt(i);
            if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')
                //if(!(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u'))
                System.out.println(ch);
        }
    }
}
```

C:\3pm>javac Test.java

C:\3pm>java Test

prakash

prakash

a

a



04. Impl prg to count numbers of vowels/consonants present in the given str.

```
-----  
import java.util.*;  
import java.util.regex.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        Scanner obj = new Scanner(System.in);  
        String s = obj.nextLine();  
        System.out.println(s);  
        int c=0;  
        for(int i=0;i<s.length();i++){  
            char ch = s.charAt(i);  
            if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')  
                c++;  
        }  
        System.out.println(c);  
    }  
}
```

C:\3pm>javac Test.java

*C:\3pm>java Test*

*welcome*

*welcome*

*3*

*05. Impl prg to sort all the characters in asc/desc order.*

```
-----  
import java.util.*;  
import java.util.regex.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        Scanner obj = new Scanner(System.in);  
        String s = obj.nextLine();  
        System.out.println(s);  
        char[] ch = s.toCharArray();  
        Arrays.sort(ch);  
        String ss = new String(ch);  
        System.out.println(ss);  
    }  
}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*welcome*

*welcome*

*ceelmow*

*06. Impl prg to check whether the given strs are anagrams or not.*

-----  
*"race" and "care"*

*4 and 4*

*['a','c','e','r'] and ['a','c','e','r']*

*import java.util.\*;*

*import java.util.regex.\*;*

*class Test*

*{*

*public static void main(String[] args)*

*{*

*String s1 = "race";*

*String s2 = "care";*

*String s3 = "cary";*

*char[] ch1 = s1.toCharArray();*

```
char[] ch2 = s2.toCharArray();

char[] ch3 = s3.toCharArray();


Arrays.sort(ch1);

Arrays.sort(ch2);

Arrays.sort(ch3);


System.out.println(Arrays.equals(ch1,ch2));//true

System.out.println(Arrays.equals(ch1,ch3));//false

    }

}
```

07. Impl prg to check whether the given str is paliandrome or not.

```
-----

import java.util.*;

import java.util.regex.*;

class Test

{

    public static void main(String[] args)

    {

        String s = "madam";

        String ss = new StringBuffer(s).reverse().toString();


        System.out.println(s.equals(ss));//true

    }

}
```

```
}
```

```
}
```

08. Impl prg to check whether the given str is pangram or not.

-----

*all english alphabets should be there in that string*

*s = "abcdefghijklmnopqrstuvwxyz"                      true*

*s = "abcdefghijklmnopqrstuvwxyz"                      false*

*s = "the quick brown fox jumps over lazy dog"    true*

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String s = "the quick brown fox jumps over lazy dog";
```

```
        boolean flag = true;
```

```
        for(int i='a';i<='z';i++){
```

```
            if(s.indexOf(i)<0)
```

```
            {
```

```
                flag=false;
```

```
                break;
```

```
        }  
    }  
    System.out.println(flag);//true  
}  
}
```

09. Impl prg to divide the strings seperated by spaces/comma/-.

```
-----  
import java.util.*;  
import java.util.regex.*;  
class Test  
{  
    public static void main(String[] args)  
    {  
        String s = "the quick brown fox jumps over lazy dog";  
        StringTokenizer st = new StringTokenizer(s);  
        while(st.hasMoreTokens())  
            System.out.println(st.nextToken());  
    }  
}
```

C:\3pm>javac Test.java

C:\3pm>java Test

*the*

*quick*

*brown*

*fox*

*jumps*

*over*

*lazy*

*dog*

10. Impl prg to reverse the entire sentence.

-----

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String s = "the quick brown fox jumps over lazy dog";
```

```
        StringBuffer sb = new StringBuffer(s);
```

```
        sb.reverse();
```

```
        System.out.println(sb);
```

```
    }
```

```
}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*god yzal revo spmuj xof nworb kciuq eht*

*11. Impl prg to reverse individual words.*

```
-----  
  
import java.util.*;  
import java.util.regex.*;  
  
class Test  
{  
  
    public static void main(String[] args)  
    {  
  
        String s = "the quick brown fox jumps over lazy dog";  
        StringTokenizer st = new StringTokenizer(s);  
        while(st.hasMoreTokens())  
            System.out.print(new StringBuffer(st.nextToken()).reverse()+" ");  
    }  
}
```

*C:\3pm>java Test*

*eht kciuq nworb xof spmuj revo yzal god*

*C:\3pm>*



12. Impl prg to reverse alternative words.

```
-----  
  
import java.util.*;  
  
import java.util.regex.*;  
  
class Test  
{  
  
    public static void main(String[] args)  
    {  
  
        String s = "the quick brown fox jumps over lazy dog";  
  
        StringTokenizer st = new StringTokenizer(s);  
  
        int i=0;  
  
        System.out.println(s);  
  
        while(st.hasMoreTokens())  
        {  
  
            if(i%2==0)  
  
                System.out.print(st.nextToken()+" ");  
  
            else  
  
                System.out.print(new StringBuffer(st.nextToken()).reverse()+" ");  
  
            i++;  
  
        }  
  
    }  
  
}
```

C:\3pm>java Test

*the quick brown fox jumps over lazy dog*

*the kciuq brown xof jumps revo lazy god*

13. Impl prg to reverse even/odd length words.

```
-----  
  
import java.util.*;  
  
import java.util.regex.*;  
  
class Test  
{  
  
    public static void main(String[] args)  
    {  
  
        String s = "the quick brown fox jumps over lazy dog";  
  
        StringTokenizer st = new StringTokenizer(s);  
  
        StringBuffer sb = new StringBuffer();  
  
        int i=0;  
  
        System.out.println(s);  
  
        while(st.hasMoreTokens())  
        {  
  
            String ss = st.nextToken();  
  
            if(ss.length()%2==0)  
  
                sb.append(new StringBuffer(ss).reverse());  
  
            else  
  
                sb.append(ss);  
  
        }  
  
    }  
}
```

```
        sb.append(" ");  
    }  
    System.out.println(sb);  
}  
}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*the quick brown fox jumps over lazy dog*

*the quick brown fox jumps revo yzal dog*

*14. Impl prg to convert every word first char into caps.*

```
-----  
import java.util.*;  
import java.util.regex.*;  
  
class Test  
{  
    public static void main(String[] args)  
    {  
        String s = "the quick brown fox jumps over lazy dog";  
        StringTokenizer st = new StringTokenizer(s);  
        StringBuffer sb = new StringBuffer();  
        System.out.println(s);  
    }  
}
```

```
while(st.hasMoreTokens())  
  
    {  
  
        String ss = st.nextToken();  
  
        sb.append(ss.substring(0,1).toUpperCase()+ss.substring(1));  
  
        sb.append(" ");  
  
    }  
  
    System.out.println(sb);  
  
}
```

*C:\3pm>java Test*

*the quick brown fox jumps over lazy dog*

*The Quick Brown Fox Jumps Over Lazy Dog*

15. Impl prg to convert every word first and last char into caps.

```
import java.util.*;  
  
import java.util.regex.*;  
  
class Test  
  
{  
  
    public static void main(String[] args)  
  
    {  
  
        String s = "the quick brown fox jumps over lazy dog";  
  
        StringTokenizer st = new StringTokenizer(s);
```

```
StringBuffer sb = new StringBuffer();

System.out.println(s);

while(st.hasMoreTokens())

{

    String ss = st.nextToken();

    int n=ss.length();

    sb.append(ss.substring(0,1).toUpperCase()+ss.substring(1,n-1)+ss.substring(n-1,n).toUpperCase());

    sb.append(" ");

}

System.out.println(sb);

}

}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*the quick brown fox jumps over lazy dog*

*ThE Quick Brown FoX JumpS Over Lazy DoG*

*16. Impl prg to convert except first and last chars, remaining into upper case.*

-----

```
import java.util.*;
```

```
import java.util.regex.*;
```

*class Test*

```
{  
  
    public static void main(String[] args)  
  
    {  
  
        String s = "the quick brown fox jumps over lazy dog";  
  
        StringTokenizer st = new StringTokenizer(s);  
  
        StringBuffer sb = new StringBuffer();  
  
        System.out.println(s);  
  
        while(st.hasMoreTokens())  
  
        {  
  
            String ss = st.nextToken();  
  
            int n=ss.length();  
  
            sb.append(ss.substring(0,1)+ss.substring(1,n-  
1).toUpperCase()+ss.substring(n-1,n));  
  
            sb.append(" ");  
  
        }  
  
        System.out.println(sb);  
  
    }  
  
}
```

*output:*

-----

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*the quick brown fox jumps over lazy dog*

*tHe qUIck bROWn fOx jUMPs oVEr lAZy dOg*

### *17. American keyboard*

-----

*Given a string, return the true if that can be typed using letters of alphabet on only one row's of American keyboard like the image below.*

*In the American keyboard:*

*=> the first row consists of the characters "qwertyuiop",*

*=> the second row consists of the characters "asdfghjkl", and*

*=> the third row consists of the characters "zxcvbnm".*

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner obj = new Scanner(System.in);
```

```
        String s = obj.nextLine();
```

```
        String r1 = "qwertyuiop";
```

```
        String r2 = "asdfghjkl";
```

```
        String r3 = "zxcvbnm";
```

```
int c1=0,c2=0,c3=0;

for(int i=0;i<s.length();i++){

    if(r1.contains(s.charAt(i)+""))

        c1++;

    if(r2.contains(s.charAt(i)+""))

        c2++;

    if(r3.contains(s.charAt(i)+""))

        c3++;

}

System.out.println(c1==s.length() || c2==s.length() || c3==s.length());

}

}
```

C:\3pm>javac Test.java

C:\3pm>java Test

mom

false

C:\3pm>java Test



*dad*

*true*

*C:\3pm>java Test*

*false*

*false*

*C:\3pm>java Test*

*true*

*true*

## 18. Rotate String

-----

*Given two strings s and ss, return true if and only if s can become ss after some number of shifts on s. A shift on s consists of moving the leftmost character of s to the rightmost position.*

*For example, if s = "abcde", then it will be "bcdea" after one shift.*

*s = "abcde"*

*"abcde"*

*"bcdea"*

*"cdeab"*

*"deabc"*

"abcd"

"bcdea" ---> true

"bdcea" ---> false

"abcdeabcde".contains(ss)

```
import java.util.*;
import java.util.regex.*;
class Test
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        String s = obj.nextLine();
        String ss = obj.nextLine();
        System.out.println((s+s).contains(ss));
    }
}
```

C:\3pm>javac Test.java

C:\3pm>java Test

abcde

*bcdea*

*true*

*C:\3pm>java Test*

*abcde*

*bdcea*

*false*

*19. Impl prg to return middle char(s).*

-----

*abc ----> b*

*abcd ---> bc*

*0123*

*4 ---> n/2-1 and n/2*

*3 ---> n/2*

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner obj = new Scanner(System.in);
```

```
        String s = obj.nextLine();
```

```
int n=s.length();

if(n%2==0)

    System.out.println(s.charAt(n/2-1)+""+s.charAt(n/2));

else

    System.out.println(s.charAt(n/2));

}

}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*abc*

*b*

*C:\3pm>java Test*

*abcd*

*bc*

*20. Impl prg to remove duplicate characters from the given str.*

-----

*"welcome" ----> "welcom"*

*"abcaba" -----> "abc"*

```
import java.util.*;
```

```
import java.util.regex.*;

class Test
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        String s = obj.nextLine();
        String ss = "";
        for(int i=0;i<s.length();i++)
        {
            if(ss.indexOf(s.charAt(i))<0)
                ss=ss+s.charAt(i);
        }
        System.out.println(ss);
    }
}
```

C:\3pm>javac Test.java

C:\3pm>java Test

welcome

welcom

C:\3pm>java Test

*abcaba*

*abc*

*C:\3pm>java Test*

*prakash*

*praksh*

## *21. Chess Board*

-----

*You are given coordinates, a string that represents the coordinates of a square of the chess board. bellow is the chess board for your reference.*

*Return True if the saquare is in white, and false if the square is in Black.*

*The coordinates will always represent a valid chess board square. The coordinates will always have the letter first, and the number second.*

```
import java.util.*;
import java.util.regex.*;
class Test
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);
        String s = obj.nextLine();
```

```
int x = s.charAt(0)-96;  
  
int y = s.charAt(1);  
  
System.out.println((x+y)%2!=0);  
  
}  
  
}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*a1*

*false*

*C:\3pm>java Test*

*a2*

*true*

*C:\3pm>java Test*

*f7*

*true*

*C:\3pm>java Test*

*h4*

*false*

22. Impl prg to convert lower case chars to upper case and vice versa (swapcase).

-----

```
import java.util.*;

import java.util.regex.*;

class Test
{
    public static void main(String[] args)
    {
        Scanner obj = new Scanner(System.in);

        String s = obj.nextLine();

        String ss="";

        for(int i=0;i<s.length();i++)
        {
            char ch = s.charAt(i);

            if(ch>='a' && ch<='z')

                ss=ss+(char)(ch-32);

            if(ch>='A' && ch<='Z')

                ss=ss+(char)(ch+32);

        }

        System.out.println(ss);

    }
}
```

C:\3pm>javac Test.java



*C:\3pm>java Test*

*PraKasH*

*pRAkASh*

*23. Impl prg to remove special characters present in the given str.*

```
-----  
  
import java.util.*;  
import java.util.regex.*;  
  
class Test  
{  
  
    public static void main(String[] args)  
    {  
  
        Scanner obj = new Scanner(System.in);  
  
        String s = obj.nextLine();  
  
        String ss="";  
  
        for(int i=0;i<s.length();i++)  
        {  
  
            char ch = s.charAt(i);  
  
            if((ch>='a'&&ch<='z') || (ch>='A'&&ch<='Z') || (ch>='0'&&ch<='9'))  
  
                ss=ss+ch;  
  
        }  
  
        System.out.println(ss);  
  
    }  
}
```

```
}
```

```
C:\3pm>javac Test.java
```

```
C:\3pm>java Test
```

```
pra$sh
```

```
prash
```

24. Impl prg to convert the given integer value into english word.

-----

```
import java.util.*;
```

```
import java.util.regex.*;
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner obj = new Scanner(System.in);
```

```
        String s = obj.nextLine();
```

```
        for(int i=0;i<s.length();i++)
```

```
        {
```

```
            char ch = s.charAt(i);
```

```
            switch(ch){
```

```
                case '0':System.out.print("zero ");break;
```

```
case '1':System.out.print("one ");break;

case '2':System.out.print("two ");break;

case '3':System.out.print("three ");break;

case '4':System.out.print("four ");break;

case '5':System.out.print("five ");break;

case '6':System.out.print("six ");break;

case '7':System.out.print("seven ");break;

case '8':System.out.print("eight ");break;

case '9':System.out.print("nine ");break;

    }

}

}
```

*C:\3pm>javac Test.java*

*C:\3pm>java Test*

*123*

*one two three*

*C:\3pm>java Test*

*56901*

*five six nine zero one*